

LothlórienGUI

Home Garden Manager



Lothlórien

Nills Maillet

13/05/2026

SLAM – BTS SIO1

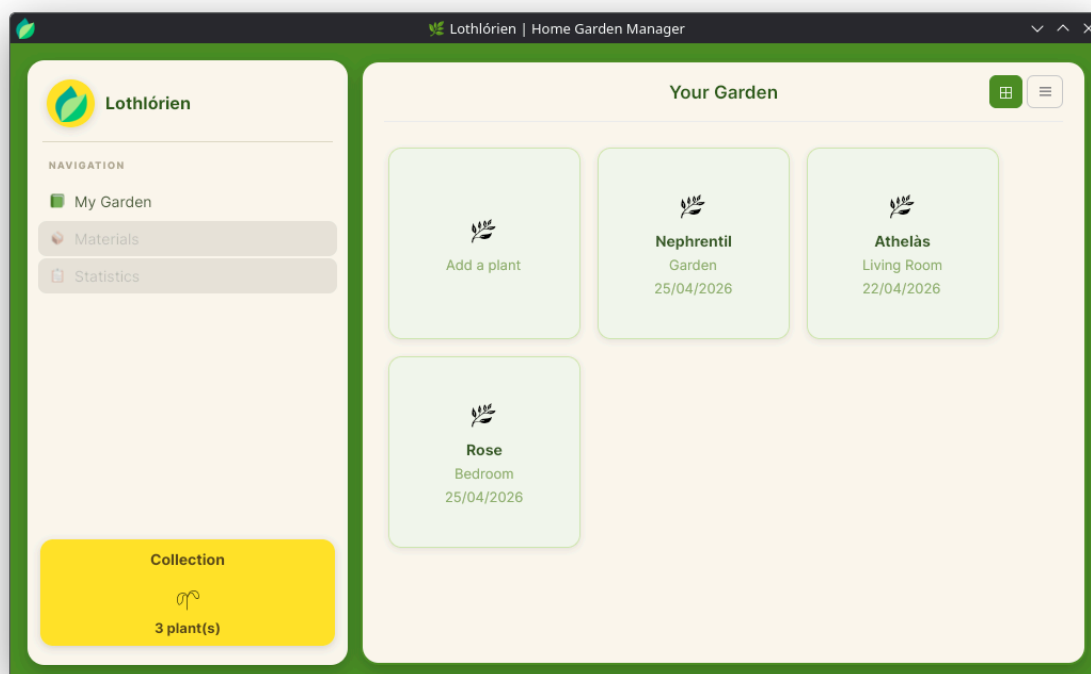
#. SOMMAIRE

#. SOMMAIRE.....	1
1. INTRODUCTION.....	2
2. OUTILS UTILISÉS.....	2
3. ARBORESCENCE DU PROJET.....	3
4. PROCÉDURE DE DÉVELOPPEMENT.....	3
4.1. Méthodologie & versioning.....	3
4.2. Étapes de développement.....	4
5. ARCHITECTURE DE L'APPLICATION.....	5
5.1. Architecture des classes.....	5
5.2. Relations entre les composants.....	6
6. FONCTIONNALITÉS RÉALISÉES.....	7
6.1. Ajout d'une plante.....	7
6.2. Affichage d'une plante.....	9
6.3. Modification d'une plante.....	12
6.4. Sauvegarde des données.....	14
6.5. Lecture des données.....	15
6.6. Affichage des données.....	16
7. CONCLUSION & PERSPECTIVES.....	17
8. RÉFÉRENCES.....	18
8.1. Documentations.....	18
8.2. Outils.....	18

1. INTRODUCTION

LothlorienGUI est une application de bureau compatible Windows et Linux dont l'objectif est de faciliter la gestion d'un jardin personnel. Elle permet notamment d'inventorier ses plantes et de consulter leurs informations détaillées. Des fonctionnalités supplémentaires, telles que la gestion de matériel et le suivi de l'arrosage, sont prévues dans de futures versions.

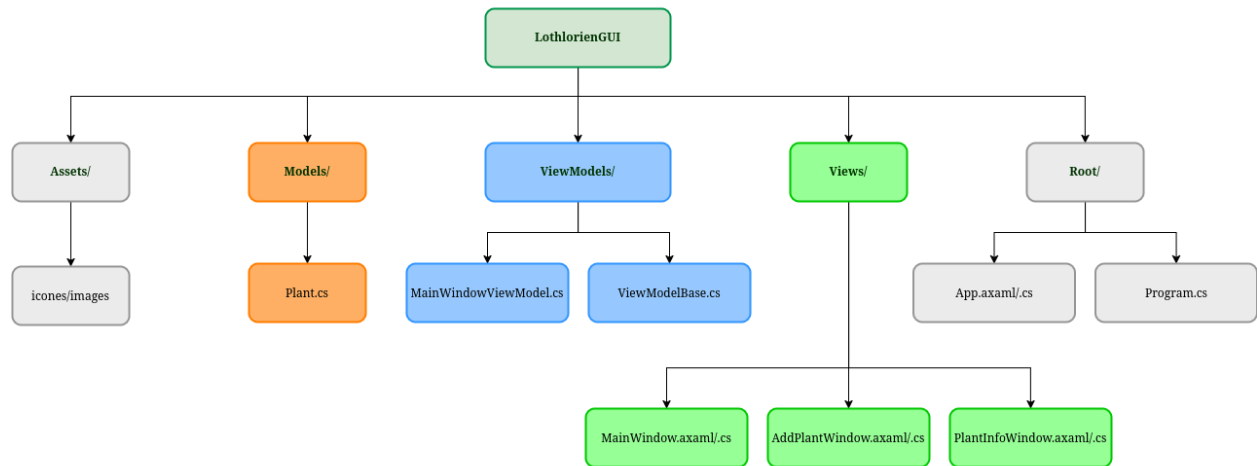
Le repository associé est disponible sur github au lien suivant : [LothlorienGUI](#).



2. OUTILS UTILISÉS

Le projet est développé en C# avec le SDK .NET 10. L'interface graphique est construite avec AvaloniaUI, un framework open-source permettant de créer des applications de bureau multiplateformes (Windows, Linux, macOS) à partir d'un code unique, en utilisant un système de balisage XAML adapté nommé AXAML. Le développement a été réalisé sous JetBrains Rider, un IDE dédié aux projets .NET, et le versioning du code est assuré par Git.

3. ARBORESCENCE DU PROJET



Le projet suit la structure standard d'une application Avalonia UI. Les vues (fichiers **.axaml** et leur code-behind **.axaml.cs**) sont regroupées dans le dossier **Views/**, les modèles de données dans **Models/**, et les ViewModels dans **ViewModels/**. Les ressources graphiques (icônes, images) sont placées dans **Assets/**.

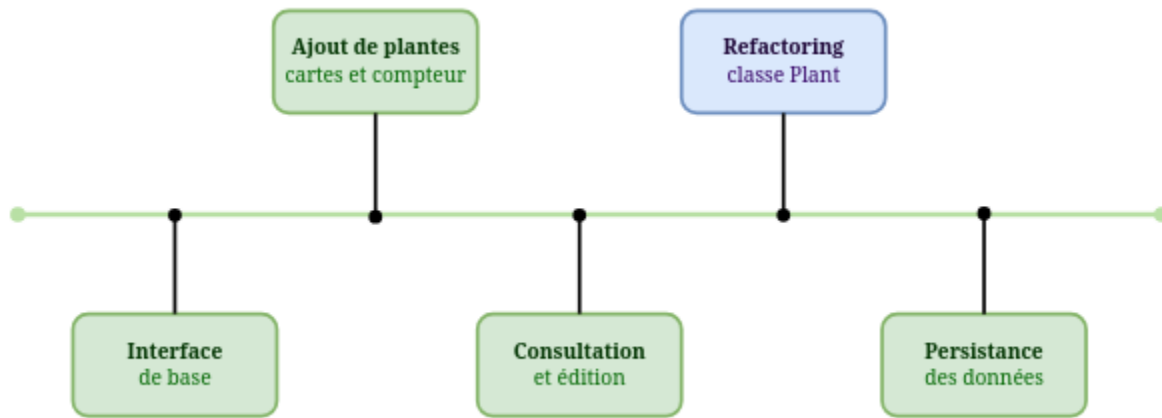
4. PROCÉDURE DE DÉVELOPPEMENT

4.1. Méthodologie & versioning

Le développement a été organisé de manière séquentielle, en traitant une fonctionnalité à la fois avant de passer à la suivante. Cette approche a permis de garder une base de code stable tout au long du projet.

Pour le suivi des modifications, Git a été utilisé avec un dépôt distant hébergé sur GitHub. Afin de garder un historique lisible, les commits suivent la convention *Conventional Commits* : chaque message est préfixé selon la nature du changement, **feat:** pour une nouvelle fonctionnalité, **refactor:** pour une restructuration du code, **fix:** pour une correction, ou encore **style:** pour des retouches visuelles.

4.2. Étapes de développement



Le développement a débuté par la mise en place de l'interface principale de l'application et d'un premier formulaire permettant d'ajouter une plante. Dans la continuité, ce formulaire a été enrichi de nouveaux champs de saisie, et l'affichage des plantes ajoutées sous forme de cartes visuelles ainsi qu'un compteur ont été intégrés à l'interface.

L'étape suivante a consisté à développer une fenêtre dédiée à la consultation et à la modification des informations d'une plante. Cette phase a également été l'occasion d'affiner l'apparence générale de l'application et de centraliser les ressources graphiques pour en faciliter la maintenance.

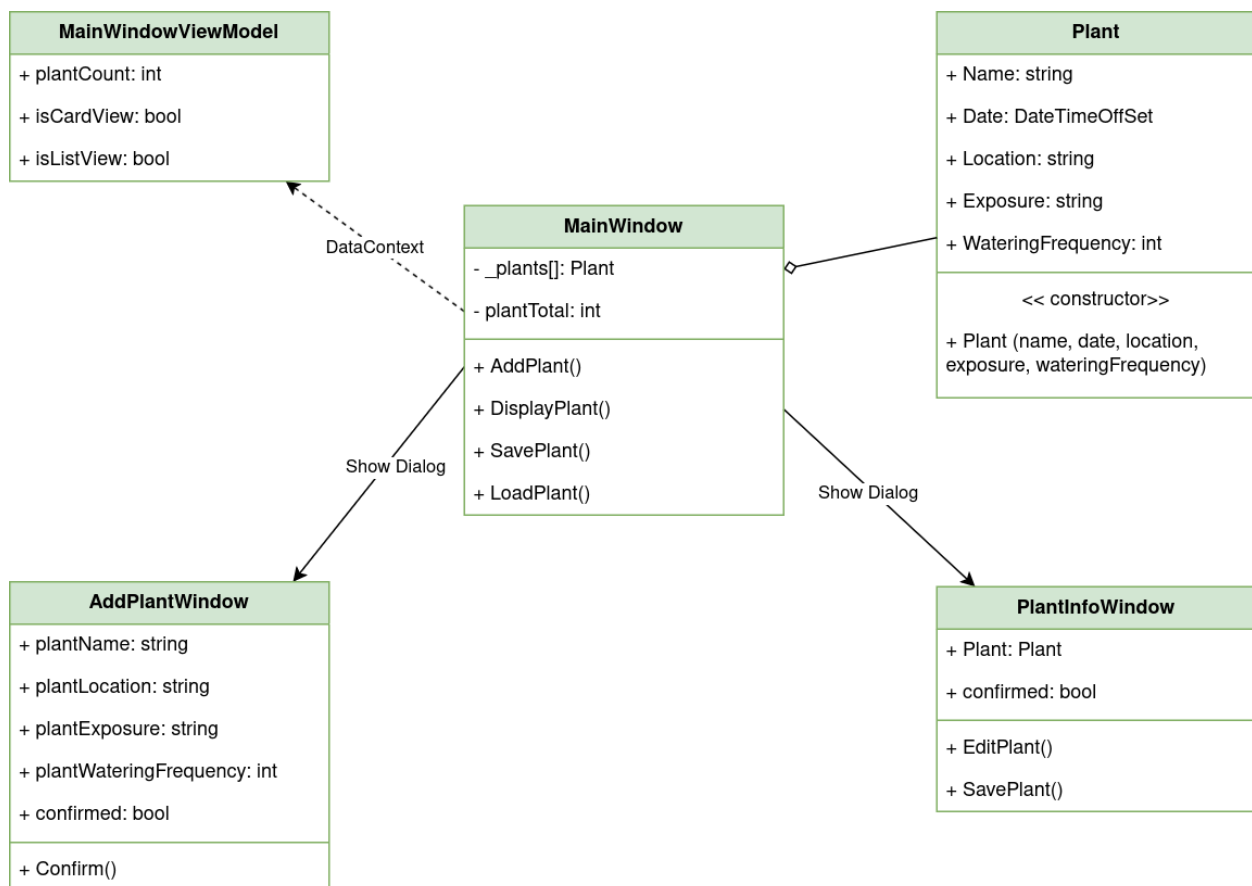
Un refactoring important a ensuite été mené : les données de chaque plante, jusqu'alors dispersées dans plusieurs tableaux parallèles distincts, ont été regroupées au sein d'une classe dédiée. Cette restructuration a amélioré significativement la lisibilité et la maintenabilité du code.

Enfin, une fonctionnalité de persistance des données a été ajoutée, permettant à l'application de sauvegarder les plantes entre les sessions et de les recharger automatiquement au démarrage.

5. ARCHITECTURE DE L'APPLICATION

5.1. Architecture des classes

Le diagramme ci-dessous représente les principales classes de l'application ainsi que leurs relations. Il se concentre sur la logique métier et les responsabilités de chaque composant, les détails d'implémentation propres à Avalonia UI, tels que les gestionnaires d'événements ou les méthodes purement graphiques, ont été volontairement omis pour des raisons de lisibilité.



5.2. Relations entre les composants

La classe Plant constitue le modèle central de l'application. Elle regroupe toutes les données relatives à une plante et est instanciée par MainWindow lors de l'ajout d'une nouvelle entrée.

MainWindow joue un rôle central dans l'architecture : elle orchestre l'ensemble des interactions, maintient la collection de plantes en mémoire et gère la persistance des données. Elle s'appuie sur MainWindowViewModel pour piloter l'état de l'interface via une liaison de données.

AddPlantWindow et PlantInfoWindow sont des fenêtres secondaires ouvertes à la demande. La communication repose sur un mécanisme synchrone : ShowDialog bloque l'exécution jusqu'à la fermeture de la fenêtre, après quoi MainWindow consulte les propriétés publiques exposées pour récupérer le résultat.

La liaison avec MainWindowViewModel fonctionne différemment, via le DataContext d'Avalonia. Lorsque PlantCount ou IsCardView sont modifiés, l'interface se met à jour automatiquement sans appel direct, grâce au mécanisme INotifyPropertyChanged fourni par le CommunityToolkit.Mvvm.

6. FONCTIONNALITÉS RÉALISÉES

6.1. Ajout d'une plante

Lorsque l'utilisateur clique sur le bouton d'ajout, une fenêtre de saisie s'ouvre en mode dialogue. À sa fermeture, si l'ajout est confirmé, une nouvelle instance de **Plant** est créée à partir des données saisies, ajoutée au tableau `_plants[]`, et une carte visuelle est générée dynamiquement dans l'interface. Les données sont ensuite sauvegardées automatiquement.

```
private async void AddOnPlantButton_OnClick(object? sender, RoutedEventArgs e)
{
    var plantInputWindow = new AddPlantWindow();

    await plantInputWindow.ShowDialog(this);

    if (plantInputWindow.confirmed)
    {
        _plants[plantTotal] = new Plant(
            plantInputWindow.plantName,
            plantInputWindow.purchaseDate ?? DateTimeOffset.Now,
            plantInputWindow.plantLocation,
            plantInputWindow.plantExposure,
            plantInputWindow.wateringFrequency);

        plantTotal++;

        if (DataContext is MainWindowViewModel vm)
            vm.PlantCount = plantTotal;

        CardPanel.Children.Add(CreatePlantCard(_plants[plantTotal - 1]));

        SavePlant();
    }
}
```


Le formulaire d'ajout permet de renseigner le nom, la date d'achat, l'emplacement, l'exposition et la fréquence d'arrosage de la nouvelle plante.

The image shows a mobile application interface for managing a plant collection. A central modal window titled "Add a new plant" is open, displaying a form to add a new plant. The form includes the following fields and options:

- Plant name/variety :** A text input field.
- Purchase date :** A date picker with fields for month, day, and year.
- Placement :** A dropdown menu with the option "Choose a location".
- Exposure level :** Three radio button options represented by icons: a sun (full sun), a sun with a cloud (partial sun), and a cloud (no sun).
- Watering frequency :** A dropdown menu showing "Every 7 days" with up and down arrows for adjustment.
- Save and add plant** : A green button at the bottom of the form.

The background of the app shows a sidebar with the following elements:

- Lothlórion** : The app's logo and name.
- NAVIGATION** : A list of options: "My Garden", "Materials", and "Statistics".
- Collection** : A yellow button with a plant icon and the text "3 plant(s)".

On the right side of the background, there is a card for a plant named "Athelàs" located in the "Living Room" with a purchase date of "22/04/2026".

6.2. Affichage d'une plante

Un clic sur une carte déclenche l'ouverture de **PlantInfoWindow**, à laquelle l'objet Plant correspondant est passé en paramètre. Le constructeur de la fenêtre initialise tous les champs avec les données de la plante. À la fermeture, si une modification a été confirmée, **MainWindow** met à jour l'objet Plant avec les nouvelles valeurs tout en la mettant à jour dans le fichier de sauvegarde grâce à `UpdatePlant()`.

```
private async void DisplayPlantInfo_OnClick(object? sender, RoutedEventArgs e)
{
    var button = sender as Button;
    var plant = (Plant)button?.Tag;

    var plantInfoWindow = new PlantInfoWindow(plant);

    await plantInfoWindow.ShowDialog(this);

    if (plantInfoWindow.confirmed)
    {
        plant.Name = plantInfoWindow.Plant.Name;
        plant.Date = plantInfoWindow.Plant.Date;
        plant.Location = plantInfoWindow.Plant.Location;
        plant.Exposure = plantInfoWindow.Plant.Exposure;
        plant.WateringFrequency = plantInfoWindow.Plant.WateringFrequency;

        var oldCard = button?.Parent as Border;
        int cardIndex = CardPanel.Children.IndexOf(oldCard);
        CardPanel.Children.RemoveAt(cardIndex);
        CardPanel.Children.Insert(cardIndex, CreatePlantCard(plant));

        UpdatePlant(plant);
    }
}
```

Le constructeur de **PlantInfoWindow** initialise chaque champ de l'interface à partir des données reçues. Pour la liste déroulante et les boutons radio, une correspondance manuelle est effectuée puisque ces contrôles ne supportent pas de liaison de données directe dans ce contexte.

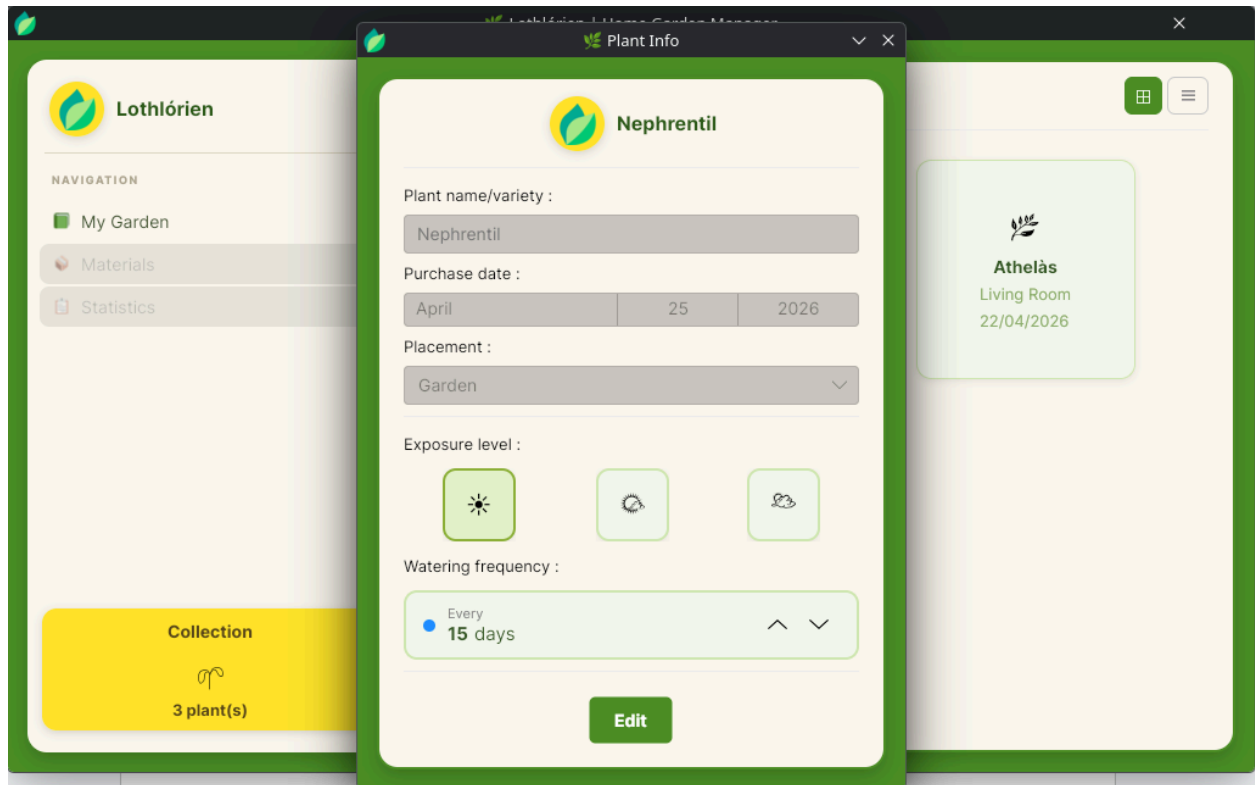
```
public PlantInfoWindow(Plant plant)
{
    InitializeComponent();
    this.Plant = plant;
    InfoTitleTextBlock.Text = plant.Name;
    PlantNameTextBox.Text = plant.Name;
    PlantDatePicker.SelectedDate = plant.Date;

    foreach (var item in PlantLocationComboBox.Items)
    {
        var comboItem = item as ComboBoxItem;
        if (comboItem?.Content?.ToString() == plant.Location)
        {
            PlantLocationComboBox.SelectedItem = comboItem;
            break;
        }
    }

    if (plant.Exposure == "FullSun")
    {
        FullSunRadioButton.IsChecked = true;
    } else if (plant.Exposure == "PartialSun")
    {
        PartialSunRadioButton.IsChecked = true;
    } else if (plant.Exposure == "LowSun")
    {
        LowSunRadioButton.IsChecked = true;
    }

    WateringFrequencyInput.Value = plant.WateringFrequency;
}
```

Un clic sur une carte ouvre la fiche détaillée de la plante en mode lecture.



6.3. Modification d'une plante

La modification s'effectue en deux temps depuis la fenêtre **PlantInfoWindow**. Par défaut, les champs sont affichés en lecture seule. Un clic sur le bouton "Edit" déverrouille les champs et bascule l'interface en mode édition.

```
private void EditPlantButton_OnClick(object? sender, RoutedEventArgs e)
{
    PlantNameTextBox.IsEnabled = true;
    PlantDatePicker.IsEnabled = true;
    PlantLocationComboBox.IsEnabled = true;
    FullSunRadioButton.IsHitTestVisible = true;
    PartialSunRadioButton.IsHitTestVisible = true;
    LowSunRadioButton.IsHitTestVisible = true;
    WateringFrequencyInput.IsHitTestVisible = true;

    EditPlantButton.IsVisible = false;
    SavePlantButton.IsVisible = true;
}
```

Une fois les modifications effectuées, un clic sur "Save" applique les nouvelles valeurs directement sur l'objet Plant, marque la modification comme confirmée et ferme la fenêtre. **MainWindow** récupère ensuite l'objet mis à jour.

```
private void SavePlantButton_OnClick(object? sender, RoutedEventArgs e)
{
    Plant.Name = PlantNameTextBox.Text;
    Plant.Date = (DateTimeOffset)PlantDatePicker.SelectedDate;
    Plant.Location = (PlantLocationComboBox.SelectedItem as
    ComboBoxItem)?.Content?.ToString() ?? "";

    if (FullSunRadioButton.IsChecked == true)
        Plant.Exposure = "FullSun";
    else if (PartialSunRadioButton.IsChecked == true)
        Plant.Exposure = "PartialSun";
    else if (LowSunRadioButton.IsChecked == true)
        Plant.Exposure = "LowSun";
}
```

```
Plant.WateringFrequency = (int)(WateringFrequencyInput.Value ?? 7);

confirmed = true;

Close();
}
```

Une fois la fenêtre fermée, la méthode UpdatePlant() se déclenche et remplace l'ancienne plante par celle mis à jour dans le fichier de sauvegarde.

```
private void UpdatePlant(Plant plant)
{
    string[] lines = File.ReadAllLines(filePath);
    int index = Array.IndexOf(_plants, plant);

    lines[index] = plant.Name + "," +
        plant.Date.ToString("yyyy-MM-dd") + "," +
        plant.Location + "," +
        plant.Exposure + "," +
        plant.WateringFrequency;

    File.WriteAllLines(filePath, lines);
}
```

6.4. Sauvegarde des données

À chaque ajout de plante, les données sont persistées dans un fichier texte `plantsData.txt` via la méthode `SavePlant()`. Chaque plante est enregistrée sur une ligne, avec ses propriétés séparées par des virgules.

```
private void SavePlant()
{
    using (StreamWriter writer = new StreamWriter(filePath, true))
    {
        writer.WriteLine(
            _plants[plantTotal - 1].Name + "," +
            _plants[plantTotal - 1].Date.ToString("yyyy-MM-dd") + "," +
            _plants[plantTotal - 1].Location + "," +
            _plants[plantTotal - 1].Exposure + "," +
            _plants[plantTotal - 1].WateringFrequency);
    }
}
```

Les données seront ainsi écrites dans le fichier **plantsData.txt** de cette manière :

```
Nephrentil,2026-04-25,Garden,FullSun,15
Athelàs,2026-04-22,Living Room,FullSun,4
Rose,2026-04-25,Bedroom,FullSun,4
```

6.5. Lecture des données

Au démarrage de l'application, la méthode **LoadPlants()** est appelée à l'ouverture de la fenêtre principale. Elle lit le fichier **plantsData.txt** ligne par ligne, reconstruit chaque **Plant** en parsant les valeurs séparées par des virgules, et génère dynamiquement les cartes correspondantes dans l'interface.

```
private void LoadPlants()
{
    if (File.Exists(filePath))
    {
        foreach (string line in File.ReadAllLines(filePath))
        {
            string[] plant = line.Split(",");

            _plants[plantTotal] = new Plant(
                plant[0],
                DateTimeOffset.Parse(plant[1]),
                plant[2],
                plant[3],
                int.Parse(plant[4])
            );

            plantTotal++;

            CardPanel.Children.Add(CreatePlantCard(_plants[plantTotal - 1]));
        }

        if (DataContext is MainWindowViewModel vm)
            vm.PlantCount = plantTotal;
    }
}
```


6.6. Affichage des données

Chaque plante est représentée par une carte visuelle générée dynamiquement par la méthode **CreatePlantCard()**. Cette méthode construit l'ensemble des éléments graphiques par code sans AXAML et les assemble dans un Border cliquable ajouté au WrapPanel de la fenêtre principale. La référence à l'objet Plant est stockée dans le Tag du bouton, ce qui permet de le récupérer lors d'un clic.

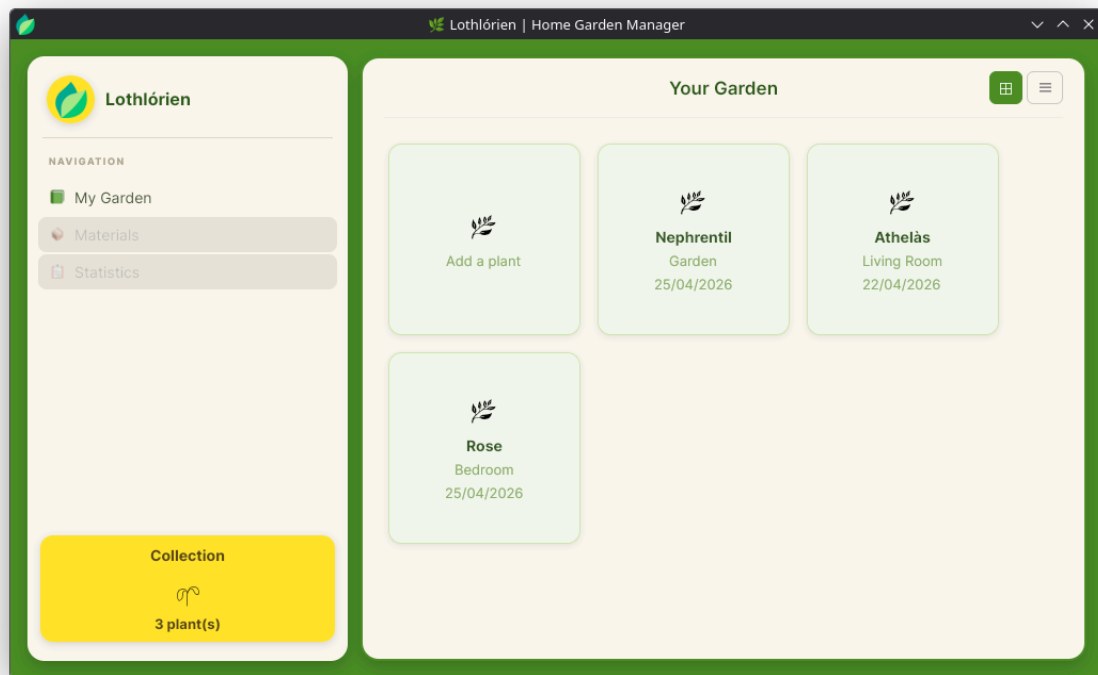
```
private Border CreatePlantCard(Plant plant)
{
    var plantCard = new Border();
    plantCard.Width = 180;
    plantCard.Height = 180;
    plantCard.Margin = new Thickness(8);
    plantCard.CornerRadius = new CornerRadius(12);
    plantCard.Background = new SolidColorBrush(Color.Parse("#F4F9EE"));
    plantCard.BorderBrush = new SolidColorBrush(Color.Parse("#D0E8B0"));
    plantCard.BorderThickness = new Thickness(1);
    plantCard.BoxShadow = BoxShadows.Parse("0 2 8 0 #14000000");

    var plantCardButton = new Button();
    //...

    plantCardButton.Tag = plant;

    return plantCard;
}
```

Rendu sur l'application :



7. CONCLUSION & PERSPECTIVES

LothlóríenGUI remplit les objectifs fixés pour cette première version : il est possible d'ajouter des plantes, de consulter et modifier leurs informations, et de persister les données entre les sessions. L'application constitue une base fonctionnelle solide sur laquelle il est possible de continuer à construire.

Plusieurs améliorations sont néanmoins prévues. Sur le plan des fonctionnalités, la vue en liste, les statistiques et la gestion du matériel sont planifiées.

8. RÉFÉRENCES

8.1. Documentations

1. [AvaloniaUI Documentation](#)
2. [Microsoft C# Classes Documentation](#)

8.2. Outils

1. [Diagrams.net](#)
2. [Github.com](#)